Logical Foundations of Concurrent Computation

Jorge A. Pérez
University of Groningen, The Netherlands
j.a.perez [[at]] rug.nl

University of Oldenburg
March 9, 2026

This Talk

A brief introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

This Talk

A brief introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.

This Talk

A brief introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.
- A disciplined usage of sessions is key to ensure program correctness.
  Not only: sessions have elegant **logical foundations**!

This Talk

A brief introduction to **session types** for concurrency, and to the logical foundations of message-passing computation.

- In a nutshell, a **session** provides a structure for a series of related interactions between communicating programs.
- A disciplined usage of sessions is key to ensure program correctness.
  Not only: sessions have elegant **logical foundations**!
- **Linear Logic**: A resource-oriented view on propositions / session protocols.
  Linear resources can be used exactly once.

Concurrent Programs

Message-Passing Concurrent Programs

- Software components (**services**) distributed across networks

Message-Passing Concurrent Programs

- Software components (**services**) distributed across networks
- Coordination takes place by sending and receiving messages

Message-Passing Concurrent Programs

- Software components (**services**) distributed across networks
- Coordination takes place by sending and receiving messages
- Compatible message exchanges are crucial for system correctness

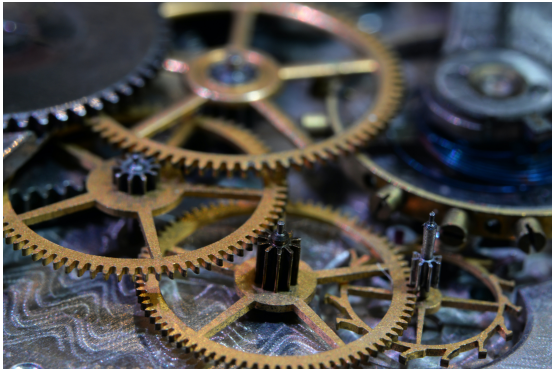
Message-Passing Concurrent Programs

- Software components (**services**) distributed across networks
- Coordination takes place by sending and receiving messages
- Compatible message exchanges are crucial for system correctness
- Even a single faulty exchange can cause system-wide bugs

Message-Passing Concurrent Programs

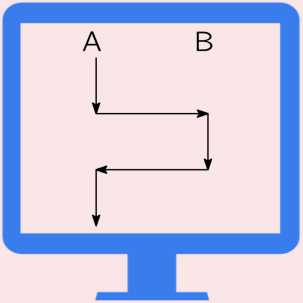
- Software components (**services**) distributed across networks
- Coordination takes place by sending and receiving messages
- Compatible message exchanges are crucial for system correctness
- Even a single faulty exchange can cause system-wide bugs

An (imperfect) analogy:



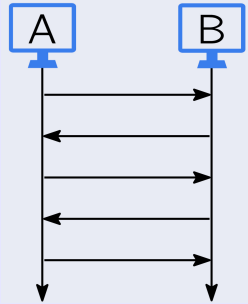
When is a Program Correct?

Sequential Programs



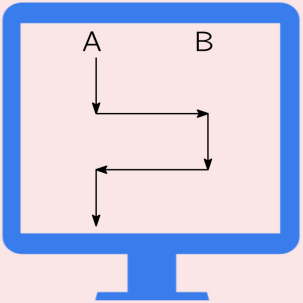
"Programs produce outputs that are consistent with their input"

Concurrent Programs



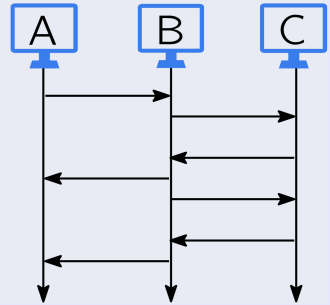
When is a Program Correct?

Sequential Programs



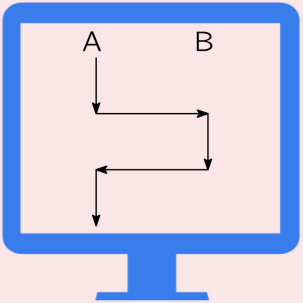
"Programs produce outputs that are consistent with their input"

Concurrent Programs



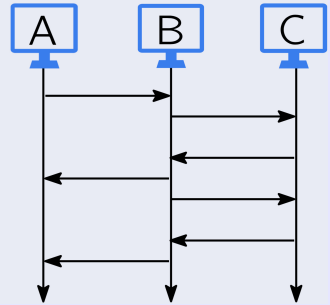
When is a Program Correct?

Sequential Programs



"Programs produce outputs that are consistent with their input"

Concurrent Programs



"Programs always respect their intended **protocols**"

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller**

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
   a) completing the payment and arranging delivery details
   b) canceling the transaction

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
   a) completing the payment and arranging delivery details
   b) canceling the transaction
4. In case(a) Alice obtains Bob's address, and sends it to Seller.

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
   a) completing the payment and arranging delivery details
   b) canceling the transaction
4. In case(a) Alice obtains Bob's address, and sends it to Seller.

4'. In case (b) Alice gracefully concludes the conversation.

Example: A Two-Buyer Protocol

Alice and **Bob** cooperate in buying a book from **Seller** :

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
   a) completing the payment and arranging delivery details
   b) canceling the transaction
4. In case(a) Alice obtains Bob's address, and sends it to Seller.

4'. In case (b) Alice gracefully concludes the conversation.

Note: The structure of messaging matters!

Example: A Two-Buyer Protocol

Desiderata for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
  - Alice never ask Bob twice within the same conversation
  - Alice doesn't continue the transaction if Bob can't contribute
  - Alice chooses among the options provided by Seller

Example: A Two-Buyer Protocol

Desiderata for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
  - Seller always returns an integer when Alice requests a quote

Example: A Two-Buyer Protocol

Desiderata for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock freedom** – they do not **"get stuck"** while running the protocol.
  - Alice eventually receives an answer from Bob on his share.

Example: A Two-Buyer Protocol

Desiderata for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock freedom** – they do not **"get stuck"** while running the protocol.
- **Termination** – they do not engage in **infinite behavior** (that may prevent them from completing the protocol)

Example: A Two-Buyer Protocol

Desiderata for the implementations of Alice, Bob, and Seller:

- **Fidelity** – they **follow the intended protocol**.
- **Safety** – they don't feature **communication errors**.
- **Deadlock freedom** – they do not **"get stuck"** while running the protocol.
- **Termination** – they do not engage in **infinite behavior** (that may prevent them from completing the protocol)

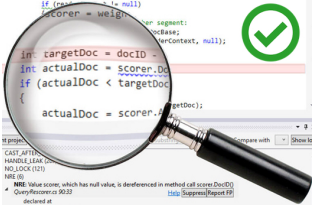
Correctness

- A **non-trivial notion**, which follows from the interplay of these properties.
- **Hard to enforce** due to actions being "scattered around" in programs.

→ A session specifies a protocol's structure, enabling program verification. It stipulates **what** and **when** should be exchanged (along a channel)

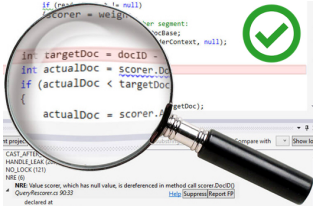
Type Systems

- Can detect bugs before programs are run
- Attached to many programming languages
- Implement a specific notion of correctness
  A program is either correct or incorrect



Type Systems

- Can detect bugs before programs are run
- Attached to many programming languages
- Implement a specific notion of correctness
  A program is either correct or incorrect



Sequential Languages

- **Data** type systems classify values in a program
- Examples: Integers, strings, etc

Type Systems

- Can detect bugs before programs are run
- Attached to many programming languages
- Implement a specific notion of correctness
  A program is either correct or incorrect

Sequential Languages

- **Data** type systems classify values in a program
- Examples: Integers, strings, etc

Concurrent Languages

- **Behavioral** type systems classify protocols in a program
- Example: “first send username, then receive true/false, finally close”
- A typical bug: sending messages in the wrong order

Protocols as Session Types

Session types specify protocols in terms of

- communication actions: send and receive
- choices: offers and selections
- recursion

Protocols as Session Types

Session types specify protocols in terms of

- communication actions: send and receive
- choices: offers and selections
- recursion

Session protocols are attached to **interaction devices**:

- communication channels in programs (think Go and Rust)
- TCP-IP sockets
- ...

Protocols as Session Types

A formal syntax for protocols:

$S ::= \ !U; S$ **send value** of type U, continue as S

Protocols as Session Types

A formal syntax for protocols:

$$
\begin{array}{rcll}
S & ::= & !\,U\,;\,S & \textbf{send value} \text{ of type } U\text{, continue as } S \\
 & \mid & ?\,U\,;\,S & \textbf{receive value} \text{ of type } U\text{, continue as } S
\end{array}
$$

Protocols as Session Types

A formal syntax for protocols:

$S ::= \; !U; S$		**send value** of type U, continue as S
$\mid \; ?U; S$		**receive value** of type U, continue as S
$\mid \; \&\{l_1 : S_1, \cdots, l_n : S_n\}$		**offer** the alternatives $S_1, \ldots, S_n$

Protocols as Session Types

A formal syntax for protocols:

$S ::= \; !U; S$		**send value** of type U, continue as S
$\mid \; ?U; S$		**receive value** of type U, continue as S
$\mid \; \&\{l_1 : S_1, \cdots, l_n : S_n\}$		**offer** the alternatives $S_1, \ldots, S_n$
$\mid \; \oplus\{l_1 : S_1, \cdots, l_n : S_n\}$		**select** one between $S_1, \ldots, S_n$

Protocols as Session Types

A formal syntax for protocols:

$S ::=$	$!U; S$	**send value** of type U, continue as S
$\mid$	$?U; S$	**receive value** of type U, continue as S
$\mid$	$\&\{l_1 : S_1, \cdots, l_n : S_n\}$	**offer** the alternatives $S_1, \ldots, S_n$
$\mid$	$\oplus\{l_1 : S_1, \cdots, l_n : S_n\}$	**select** one between $S_1, \ldots, S_n$
$\mid$	$\mu t.S \mid t$	**recursion**

Protocols as Session Types

A formal syntax for protocols:

$S ::=$	$!U; S$	**send value** of type U, continue as S
$\mid$	$?U; S$	**receive value** of type U, continue as S
$\mid$	$\&\{l_1 : S_1, \cdots, l_n : S_n\}$	**offer** the alternatives $S_1, \ldots, S_n$
$\mid$	$\oplus\{l_1 : S_1, \cdots, l_n : S_n\}$	**select** one between $S_1, \ldots, S_n$
$\mid$	$\mu t.S \mid t$	**recursion**
$\mid$	$\mathtt{end}$	**terminated protocol**

Protocols as Session Types

A formal syntax for protocols:

$S ::=$	$!U; S$	**send value** of type U, continue as S
$\mid$	$?U; S$	**receive value** of type U, continue as S
$\mid$	$\&\{l_1 : S_1, \cdots, l_n : S_n\}$	**offer** the alternatives $S_1, \ldots, S_n$
$\mid$	$\oplus\{l_1 : S_1, \cdots, l_n : S_n\}$	**select** one between $S_1, \ldots, S_n$
$\mid$	$\mu t.S \mid t$	**recursion**
$\mid$	$\mathtt{end}$	**terminated protocol**

Notice:

- Sequential communication patterns (no built-in concurrency)
- U stands for basic values (e.g. `int`) but also other sessions S

The Two-Buyer Protocol, Revisited

Recall the protocol between Alice, Bob, and Seller:

1. Alice sends a book title to Seller, who sends a quote back.
2. Alice checks whether Bob can contribute in buying the book.
3. Alice uses the answer from Bob (yes/no) to interact with Seller, either:
   a) completing the payment and arranging delivery details
   b) canceling the transaction
4. In case(a) Alice obtains Bob's address, and sends it to Seller.

4'. In case (b) Alice gracefully concludes the conversation.

Session Types for The Two-Buyer Protocol

Two independent protocols, with Alice "leading" the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{\mathsf{SA}} = ?\mathtt{book};\ !\mathtt{quote};\ \&\begin{cases}\text{buy}: & ?\mathtt{paym};\ ?\mathtt{address};\ !\mathtt{ok};\ \mathsf{end}\\ \text{cancel}: & ?\mathtt{thanks};\ !\mathtt{bye};\ \mathsf{end}\end{cases}$$

Session Types for The Two-Buyer Protocol

Two independent protocols, with Alice "leading" the interactions:

1. A session type for Seller (in its interaction with Alice):

$$S_{\mathsf{SA}} = ?\mathtt{book};\ !\mathtt{quote};\ \& \begin{cases} \mathsf{buy}: & ?\mathtt{paym};\ ?\mathtt{address};\ !\mathtt{ok};\ \mathsf{end} \\ \mathsf{cancel}: & ?\mathtt{thanks};\ !\mathtt{bye};\ \mathsf{end} \end{cases}$$

2. A session type for Bob (in its interaction with Alice):

$$S_{\mathsf{AB}} = ?\mathtt{cost};\ \oplus \begin{cases} \mathsf{share}: & !\mathtt{address};\ ?\mathtt{ok};\ \mathsf{end} \\ \mathsf{close}: & !\mathtt{bye};\ \mathsf{end} \end{cases}$$

Session Types for The Two-Buyer Protocol

Implementations for Alice, Bob, and Seller should be **compatible**.

Session Types for The Two-Buyer Protocol

Implementations for Alice, Bob, and Seller should be **compatible**.

Duality relates session types with opposite behaviors:

- The dual of sending is receiving (and vice versa)
- Branching is the dual of selection (and vice versa)

The dual of session type S is written $\overline{S}$.

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Bob's viewpoint in her interaction with Alice:

$$S_{\mathsf{AB}} = \ ?\mathtt{cost}; \oplus \begin{cases} \text{share}: & !\mathtt{address}; ?\mathtt{ok}; \text{end} \\ \text{close}: & !\mathtt{bye}; \text{end} \end{cases}$$

- Given this, Alice's implementation should conform to $\overline{S_{\mathsf{AB}}}$:

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Bob's viewpoint in her interaction with Alice:

$$S_{\mathsf{AB}} = \ ?\mathtt{cost}; \oplus \begin{cases} \text{share}: & !\mathtt{address}; ?\mathtt{ok}; \text{end} \\ \text{close}: & !\mathtt{bye}; \text{end} \end{cases}$$

- Given this, Alice's implementation should conform to $\overline{S_{\mathsf{AB}}}$:

$$\overline{S_{\mathsf{AB}}} = \ !\mathtt{cost}; \& \begin{cases} \text{share}: & ?\mathtt{address}; !\mathtt{ok}; \text{end} \\ \text{close}: & ?\mathtt{bye}; \text{end} \end{cases}$$

Session Types for The Two-Buyer Protocol

Example:

- Recall that S_{AB} describes Bob's viewpoint in her interaction with Alice:

$$S_{\mathsf{AB}} = \ ?\mathtt{cost}; \oplus \begin{cases} \text{share}: & !\mathtt{address}; ?\mathtt{ok}; \text{end} \\ \text{close}: & !\mathtt{bye}; \text{end} \end{cases}$$

- Given this, Alice's implementation should conform to $\overline{S_{\mathsf{AB}}}$:

$$\overline{S_{\mathsf{AB}}} = \ !\mathtt{cost}; \& \begin{cases} \text{share}: & ?\mathtt{address}; !\mathtt{ok}; \text{end} \\ \text{close}: & ?\mathtt{bye}; \text{end} \end{cases}$$

- That is, Alice's implementation should conform to **both** $\overline{S_{\mathsf{SA}}}$ and $\overline{S_{\mathsf{AB}}}$.

Session Type Duality, Formally

Given a (finite) session type S, its dual type $\overline{S}$ is inductively defined as follows:

$$
\begin{aligned}
\overline{!\,U;S} &= ?\,U;\overline{S} \\
\overline{?\,U;S} &= !\,U;\overline{S} \\
\overline{\&\{l_i : S_i\}_{i\in I}} &= \oplus\{l_i : \overline{S_i}\}_{i\in I} \\
\overline{\oplus\{l_i : S_i\}_{i\in I}} &= \&\{l_i : \overline{S_i}\}_{i\in I} \\
\overline{\mathsf{end}} &= \mathsf{end}
\end{aligned}
$$

Notice:

- For recursive types, duality is defined **coinductively** (the dual of $\mu t.S$ is *not* $\mu t.\overline{S}$)

Propositions as Sessions

A Curry-Howard correspondence for concurrency:

Linear logic propositions	$\leftrightarrow$	types describing **behavior** (sessions)
Sequent calculus derivations	$\leftrightarrow$	**concurrent processes** (π-calculus)
Cut reductions	$\leftrightarrow$	communication between processes

Key Ideas

- Assume a language of **processes** (denoted $P, Q, \ldots$) that interact by synchronizing on **names** (denoted $x, y, z, \ldots$).

Key Ideas

- Assume a language of **processes** (denoted $P, Q, \ldots$) that interact by synchronizing on **names** (denoted $x, y, z, \ldots$).
- Processes can send/receive messages on names, using sequencing and parallel composition. We "extract" their syntax from proofs.

Key Ideas

- Interpret the logical sequent

$$A_1, \cdots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \cdots, x_n : A_n \vdash P :: z : C$$

Process P offers session C on channel z...

Key Ideas

- Interpret the logical sequent

$$A_1, \cdots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \cdots, x_n : A_n \vdash P :: z : C$$

Process P offers session C on channel z...

... by relying on sessions $A_1, \ldots, A_n$ on channels $x_1, \ldots, x_n$

Linear Logic Propositions as Session Types

`end`	**terminate** the session	1
$!\,U\,;\,S$	**send value** of type U, continue as S	$U \otimes S$
$?\,U\,;\,S$	**receive value** of type U, continue as S	$U \multimap S$

Linear Logic Propositions as Session Types

`end`	**terminate** the session	1
$!\,U\,;\,S$	**send value** of type U, continue as S	$U \otimes S$
$?\,U\,;\,S$	**receive value** of type U, continue as S	$U \multimap S$
$S_1 \oplus S_2$	**select** one between S_1 (left) and S_2 (right)	idem
$S_1 \,\&\, S_2$	**offer** the alternatives S_1 (left) and S_2 (right)	idem

Linear Logic Propositions as Session Types

$\mathtt{end}$	**terminate** the session	1
$!\,U\,;\,S$	**send value** of type U, continue as S	$U \otimes S$
$?\,U\,;\,S$	**receive value** of type U, continue as S	$U \multimap S$
$S_1 \oplus S_2$	**select** one between S_1 (left) and S_2 (right)	idem
$S_1 \,\&\, S_2$	**offer** the alternatives S_1 (left) and S_2 (right)	idem
$\oplus\{l_1 : S_1, \cdots, l_n : S_n\}$	**select** one between $S_1, \ldots, S_n$	idem*
$\&\{l_1 : S_1, \cdots, l_n : S_n\}$	**offer** the alternatives $S_1, \ldots, S_n$	idem*

Propositions as Sessions

- A productive line of research, able to justify multiple programming phenomena from a purely logical perspective
  - Multiparty protocols, dependent/polymorphic types, client-server interactions, non-determinism, asynchronous communication, recursion, etc

Propositions as Sessions

- A productive line of research, able to justify multiple programming phenomena from a purely logical perspective
  - Multiparty protocols, dependent/polymorphic types, client-server interactions, non-determinism, asynchronous communication, recursion, etc
- Logical foundations (cut elimination) endows types processes with key properties, including deadlock freedom.
- Open challenges:
  - Stronger forms of deadlock and lock freedom by typing
  - Handling non-linear (shared) resources beyond ! (bang) in Linear Logic
  - Connections with shared-memory concurrency
  - Enhanced behavioral theories / semantic typing

Propositions as Sessions

- A productive line of research, able to justify multiple programming phenomena from a purely logical perspective
  - Multiparty protocols, dependent/polymorphic types, client-server interactions, non-determinism, asynchronous communication, recursion, etc
- Logical foundations (cut elimination) endows types processes with key properties, including deadlock freedom.
- Open challenges:
  - Stronger forms of deadlock and lock freedom by typing
  - Handling non-linear (shared) resources beyond ! (bang) in Linear Logic
  - Connections with shared-memory concurrency
  - Enhanced behavioral theories / semantic typing
  - ♡ **Coinductive session protocols and connections with proof assistants**: Our national consortium "Cyclic Structures in Programs and Proofs".